Lesson 1.7 **Duration: 40 minutes**

Recursion Basics

Learning Objectives

- Explain the concept of recursion and identify base cases
- Write recursive functions to solve mathematical and logical problems
- Compare recursive and iterative approaches for the same problem

Engage (5-7 minutes)

A Russian nesting doll contains smaller versions of itself inside. Recursion works the same way-a function calls itself to solve smaller instances of the same problem. Every recursive solution needs a stopping point, like the smallest doll that cannot be opened. Today we learn to think recursively and understand when this elegant approach is the best choice.

Explore (10-12 minutes)

Consider calculating 5! (5 factorial). Write it as a recursive definition: 5! = 5 — 4!, 4! = 4 — 3!, and so on until 0! = 1. Now write a recursive function that computes this. Trace through the calls on paper, showing the call stack. What happens if you forget the base case? Try it and observe the error.

Explain (10-12 minutes)

Recursion is when a function calls itself to solve a smaller version of the original problem. Every recursive function must have a base case-the condition that stops the recursion-and a recursive case that moves toward the base case. The call stack tracks each function call, storing local variables until the base case is reached, then unwinding with computed values. Recursion is elegant for problems like factorials, Fibonacci sequences, tree traversals, and divide-and-conquer algorithms.

Elaborate (8-10 minutes)

Implement recursive functions for: Fibonacci sequence (return the nth number), power calculation (x raised to n without using), and the Tower of Hanoi puzzle for 3 disks. For each, identify the base case and recursive case. Then write the iterative version of the factorial function and compare both approaches in terms of readability and memory usage. Discuss the concept of tail recursion and why some languages optimize it.

Evaluate (5-8 minutes)

Trace the recursive call of `fibonacci(5)` showing every function call and return value. Write the recursive function and its iterative counterpart for computing the sum of digits of a number. Explain one advantage and one disadvantage of recursion compared to iteration.